Connectivity in Directed Graphs

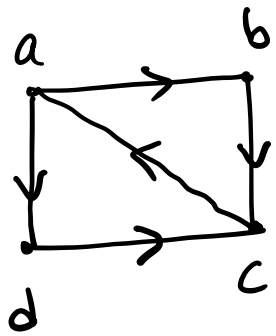
Strongly connected

Weakly connected

Def:

A directed graph is **Strongly Connected** if every two vertices are reachable from each other.

E.g..

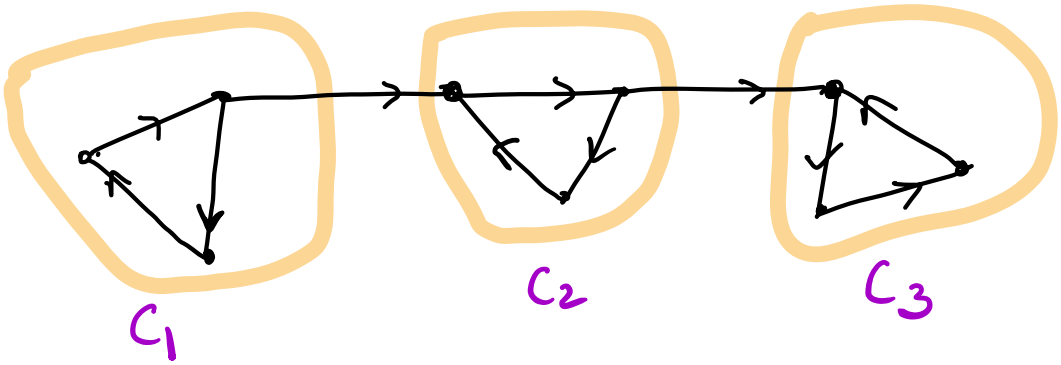


Strongly connected graph.

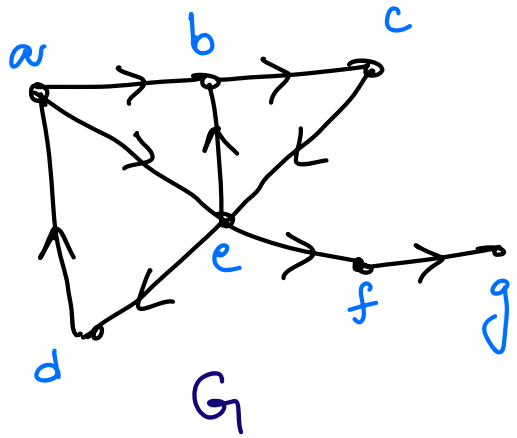
(SCC)

A **Strongly Connected Component** of a directed graph G is a subgraph that is Strongly Connected and is maximal (no additional edges or vertices can be added to subgraph without voilating the Property)

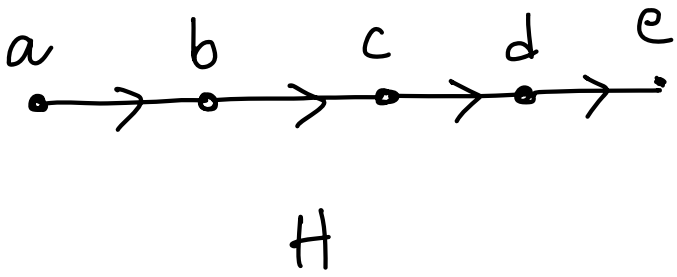
Example ② Graph with three Connected Components.



Warmup Question

① 

G

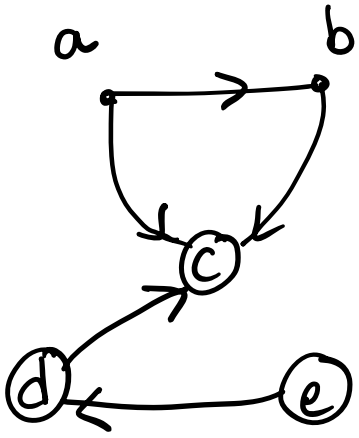


H

How many SCC's does G & H have ?

A directed graph is **Weakly Connected** if every two vertices are reachable from each other ignoring the directions of edges.

Example:

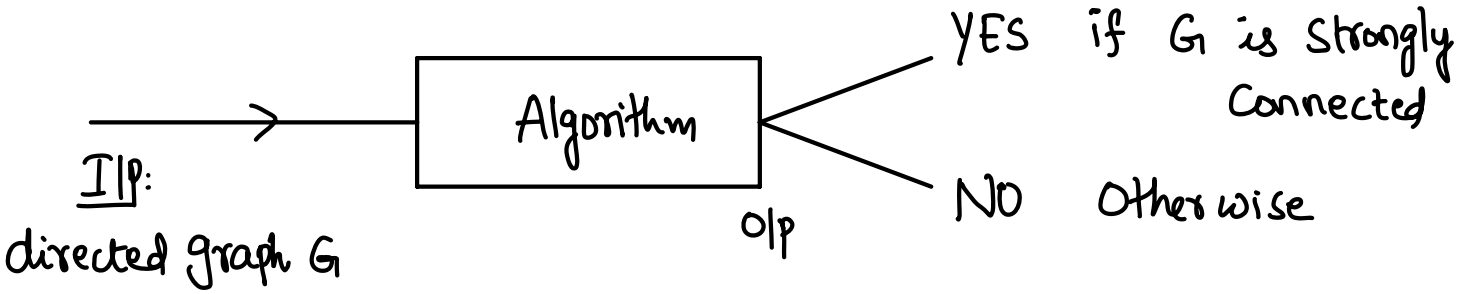


Weakly Connected.

Problem

Input: A directed graph G

Q: Check if G is Strongly connected ?



What is the trivial algorithm ?

Think about a better algorithm !

Alg - Is Strongly Connected (G)

```
1   Choose any vertex v ∈ V
2   DFS(G, v)
3   If there exists a vertex u ∈ V such that
      u is not visited then
4       return FALSE
5   Construct the transpose graph G^T
6   DFS(G^T, v)
7   If there exists a vertex u ∈ V^T such that
      u is not visited then
8       return FALSE
9   return TRUE
```

Running time: $O(n+m)$

Where n: # of vertices

m: # of edges

Next

Algorithm to find SCC's in a directed graph G.

[Slightly difficult Problem]

Problem



I/P: Directed graph G

O/P: SCC's of G

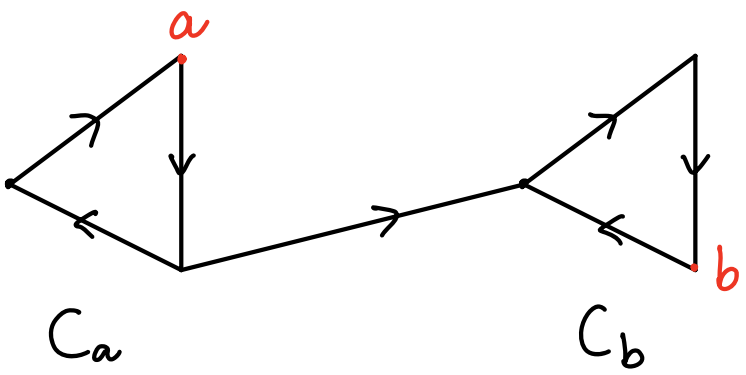
What is the trivial algorithm ?

Warmup - Ideas

TRUE|FALSE:

Let G be a directed graph with SCC's $C_1, \ldots C_k$. If we run DFS starting from a vertex $v \in C_i$ then the DFS visits only the vertices in C_i.

Warmup - Ideas



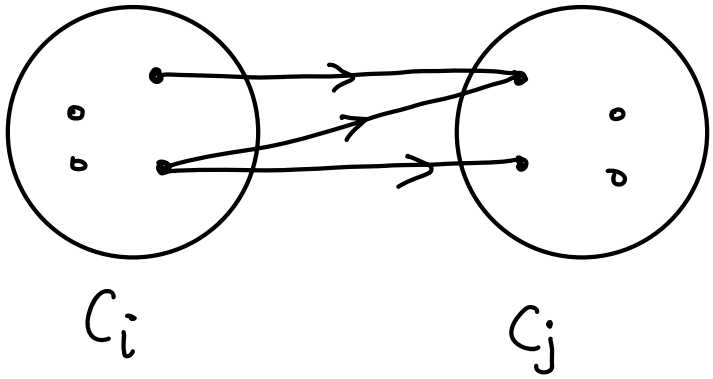
If we start DFS from node b then we explore only C_b

If we start DFS from node a then we explore both C_a and C_b

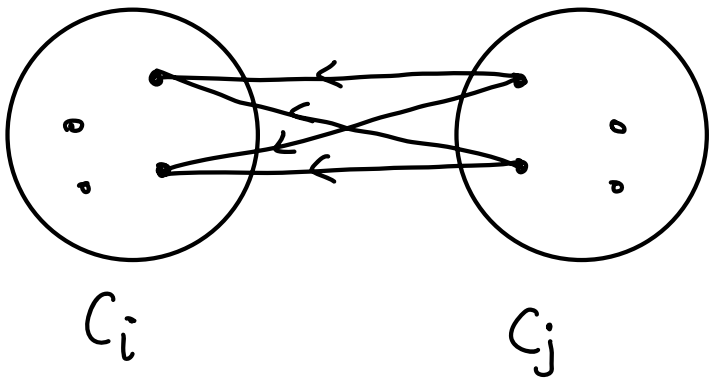
Moral: If we call the DFS from the right place then we explore only required SCC.

TRUE/FALSE:

Let G be a directed graph with SCCs $C_1, C_2, \ldots C_k$. Then, for any two distinct SCCs C_i and C_j, all edges between them go in the same direction.



or



Component graph

Construct a **Component graph** $G^{scc} = (V^{scc}, E^{scc})$

Suppose that G has Strongly Connected Components

$C_1, \ldots C_k$.

The Vertex set V^{scc} is $\{v_1, v_2, \ldots v_k\}$

ie, it contains one vertex v_i for each C_i

There is an edge $v_i v_j \in E^{scc}$ if G Contains a directed edge xy for some $x \in C_i$ and y_i in C_j.

What can you say about the graph G^{scc}?

True | False :

The graph G^{SCC} is acyclic.

Def: $G = (V, E)$

If $U \subseteq V$ then we define

$$d(U) = \min_{u \in U} \{u.d\}$$

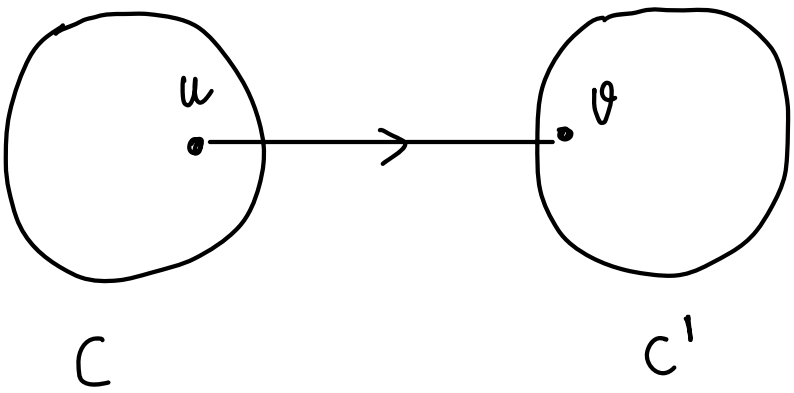
$$f(U) = \max_{u \in U} \{u.f\}$$

That is $d(U)$ and $f(U)$ are the earliest discovery time and latest finishing time respectively of any vertex in U.

Lemma:

Let C and C' be distinct Strongly Connected Components in directed graph $G = (V, E)$. Suppose that there is an edge $uv \in E$, where $u \in C$ and $v \in C'$.

Then $f(C) > f(C')$



Transpose of G : [Recap]

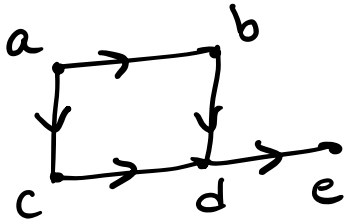
The transpose of a graph G is $G^T = (V, E^T)$

where

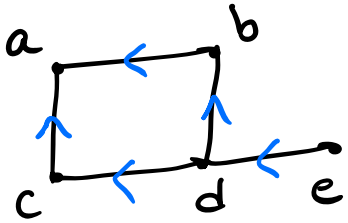
$$E^T = \{ uv \mid vu \in E(G) \}$$

ie, E^T consists of the edges of G with directions reversed.

Ex:



G



G^T

True | false

Both G and G^T have exactly the same SCCs.

ie, if u & v reachable from eachother in G if and only if they are reachable from each other in G^T

Corollary: Let C and C' be distinct connected components in directed graph $G=(V,E)$.

Suppose that there is an edge $uv \in E^T$, where $u \in C$ and $v \in C'$ then $f(C) < f(C')$

What is the final algorithm?

Algorithm: [Kosaraju]

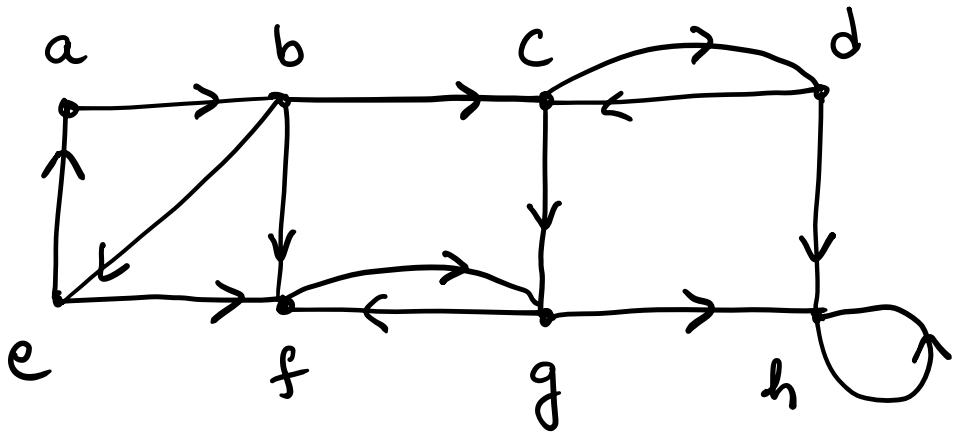
STRONGLY-CONNECTED-COMPONENTS(G)

1 call DFS(G) to compute finishing times $u.f$ for each vertex u
2 compute G^T
3 call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

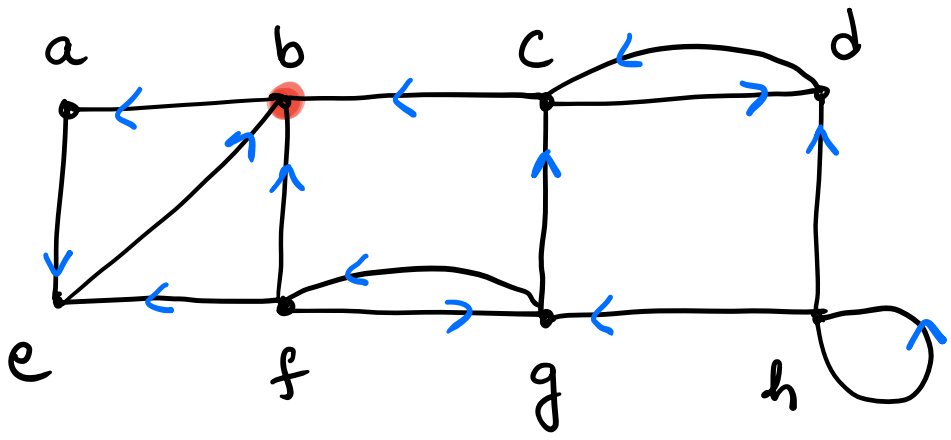
Running time: 2 * DFS running time = $O(V+E)$

Example

Graph G



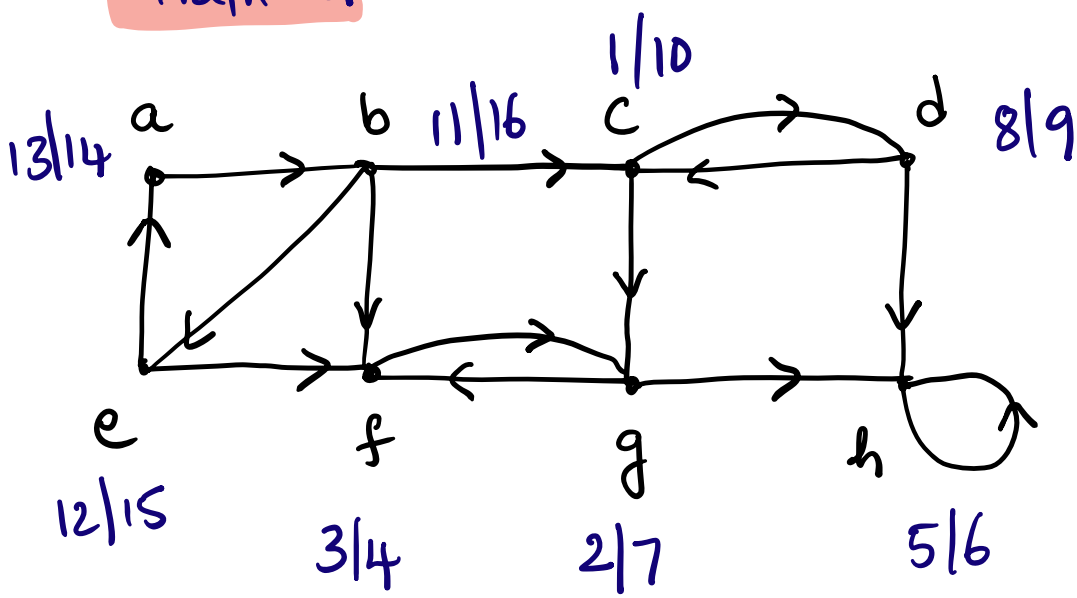
Graph G^T



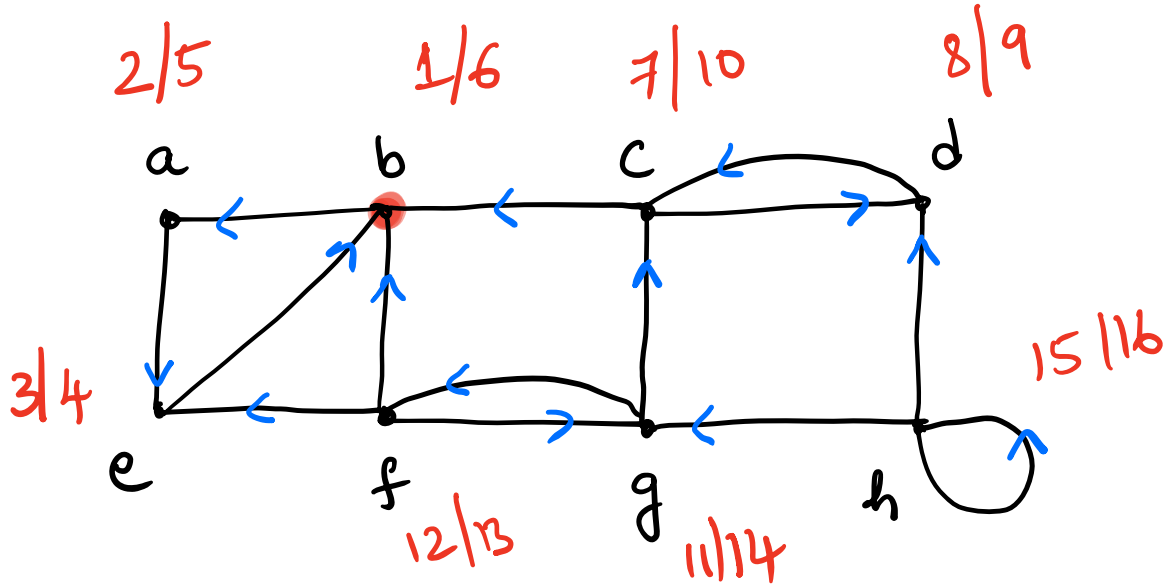
DFS forest

Example

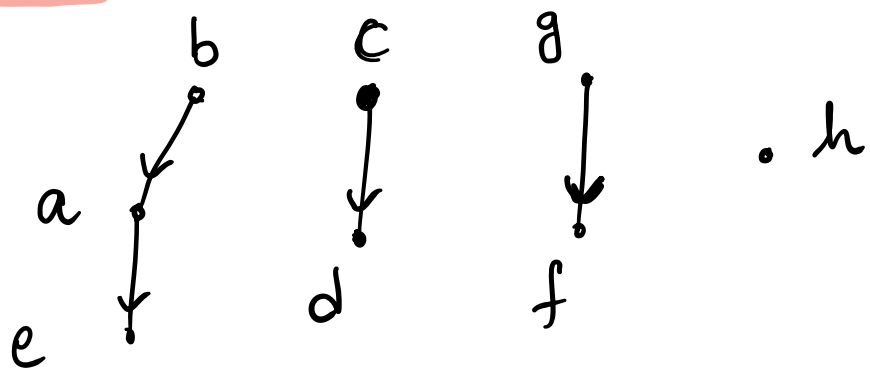
Graph G



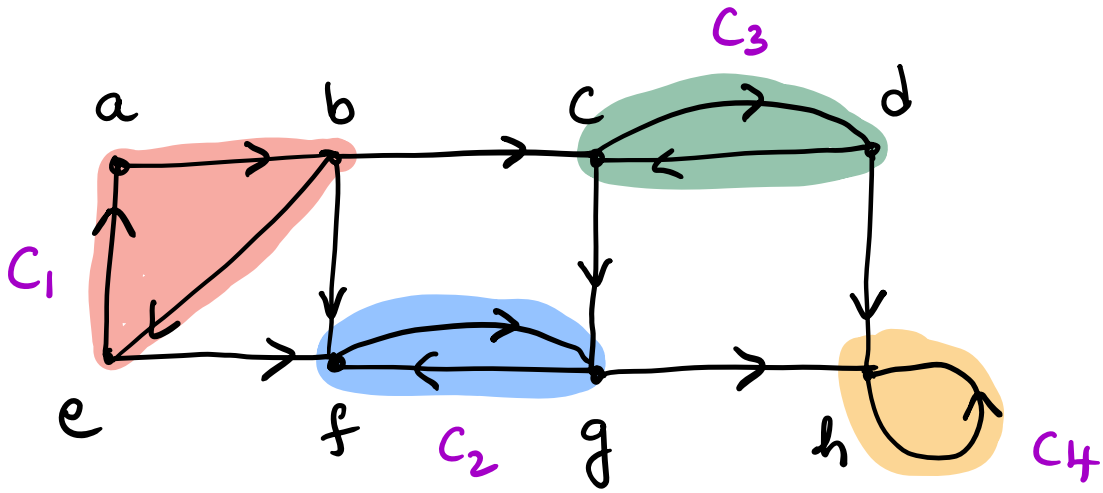
Graph G^T

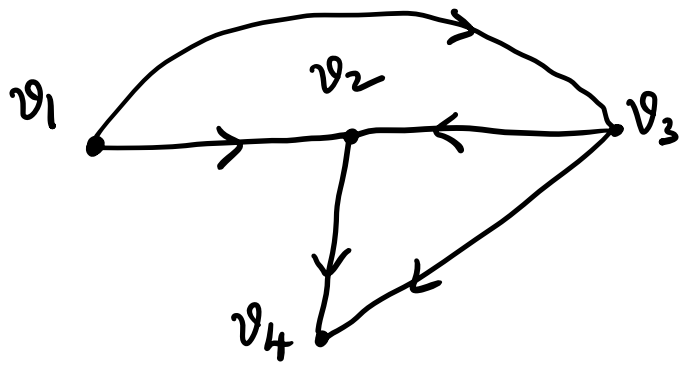


DFS forest



Four Connected Components.





Graph G^{scc}

Summary of the Algorithm

In the 2nd DFS, which is on G^T, we start with the SCC C whose finishing time $f(C)$ is maximum.

The search starts in C at some vertex x and it visits all vertices in C.

By the above corollary G^T contains no edges from C to any other SCC.

∴ The tree rooted at x contains exactly the vertices of C.

Next search selects as a root a vertex from some other SCC C' whose finishing time $f(C')$ is maximum over all components other than C.

and so on.